

AWS Deployment Guide - Parallels (Shopify SAAS)

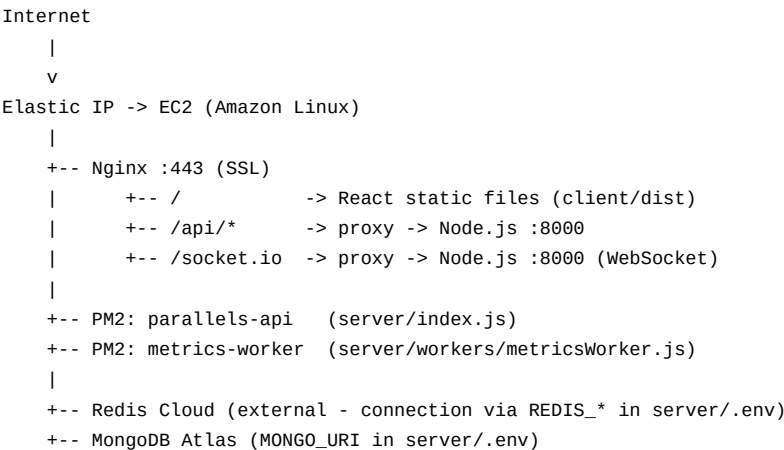
Audience: Intern / DevOps onboarding
Stack: Amazon Linux EC2 · Nginx · SSL (Let's Encrypt) · PM2 · GitHub Actions · Redis Cloud
Production domain: <https://parallels.messold.com>
API base path: All backend routes live under /api

****Important:**** This deployment uses ****Amazon Linux 2023**** on EC2. All commands below use `dnf` and the `ec2-user` account. Do not use Ubuntu unless you change the whole setup.

Table of contents

- 1. [Architecture overview](#1-architecture-overview)
- 2. [Prerequisites](#2-prerequisites)
- 3. [Create an EC2 instance](#3-create-an-ec2-instance)
- 4. [Create and use a PEM key (SSH login)](#4-create-and-use-a-pem-key-ssh-login)
- 5. [Attach an Elastic IP](#5-attach-an-elastic-ip)
- 6. [First-time server setup](#6-first-time-server-setup)
- 7. [Clone the project and install dependencies](#7-clone-the-project-and-install-dependencies)
- 8. [Environment variables](#8-environment-variables)
- 9. [PM2 - ecosystem.config.js (two processes)](#9-pm2--ecosystemconfigjs-two-processes)
- 10. [Build the frontend](#10-build-the-frontend)
- 11. [Nginx - reverse proxy, /api, and SSL](#11-nginx--reverse-proxy-api-and-ssl)
- 12. [GitHub Actions - automated deploy on push to main](#12-github-actions--automated-deploy-on-push-to-main)
- 13. [Verification checklist](#13-verification-checklist)
- 14. [Common commands](#14-common-commands)
- 15. [Troubleshooting](#15-troubleshooting)

1. Architecture overview



GitHub Actions ([.github/workflows/deploy.yml](#)) SSHs into the server on every push to main, runs git pull, npm install, and pm2 restart ecosystem.config.js.

2. Prerequisites

Before you start, make sure you have:

Item	Notes
AWS account	With permission to create EC2, Elastic IP, Security Groups
Domain name	e.g. `parallels.messold.com` - DNS must point to Elastic IP
GitHub repo access	`shopify_SAAS` repository
MongoDB connection string	Atlas `MONGO_URI`
Redis Cloud account	Hosted Redis URL / host, port, password (not installed on EC2)
`.env` secrets	Created manually on the server - GitHub Actions does not deploy these
`ecosystem.config.js`	Created manually on the server - not in GitHub (see section 9)

Recommended EC2 size: t3.medium or larger (Node + workers need RAM).

OS requirement: Use **Amazon Linux 2023** only. The deploy workflow and all setup commands assume `ec2-user` and `dnf`.

3. Create an EC2 instance

- Log in to **AWS Console** -> **EC2** -> **Launch instance**.
- Name:** `parallels-production` (or similar).
- AMI:** **Amazon Linux 2023** (required - matches `ec2-user` and `dnf` commands in this guide).
- Instance type:** `t3.medium` minimum.
- Key pair:**
 - Click Create new key pair
 - Name: parallels-prod-key
 - Type: RSA
 - Format: .pem
 - Download and store safely - you cannot download it again.
- Network settings - Security Group inbound rules:**

Type	Port	Source	Purpose
SSH	22	Your IP / office IP	SSH access
HTTP	80	0.0.0.0/0	Certbot + redirect to HTTPS
HTTPS	443	0.0.0.0/0	Public web traffic
- Storage:** 30 GB+ gp3.
- Launch the instance.

4. Create and use a PEM key (SSH login)

4.1 Fix PEM permissions (Mac / Linux)

```
chmod 400 ~/Downloads/parallels-prod-key.pem
```

4.2 SSH into the server

Replace YOUR_ELASTIC_IP with the instance public IP (or Elastic IP after step 5):

```
ssh -i ~/Downloads/parallels-prod-key.pem ec2-user@YOUR_ELASTIC_IP
```

****Username:**** Always use `ec2-user` on Amazon Linux.

4.3 Optional - SSH config (easier daily use)

Add to ~/.ssh/config on your laptop:

```
Host parallels-prod
  HostName YOUR_ELASTIC_IP
  User ec2-user
  IdentityFile ~/Downloads/parallels-prod-key.pem
```

Then connect with:

```
ssh parallels-prod
```

5. Attach an Elastic IP

An Elastic IP gives the server a fixed public IP that survives instance stop/start.

1. EC2 -> ****Elastic IPs**** -> ****Allocate Elastic IP address**** -> Allocate.
2. Select the new IP -> ****Actions**** -> ****Associate Elastic IP address****.
3. Choose your EC2 instance -> Associate.
4. Update DNS: create an ****A record**** for `parallels.messold.com` pointing to this Elastic IP.
5. Wait for DNS propagation (5-30 minutes).

After attaching Elastic IP, always SSH and configure GitHub secrets using the ****Elastic IP****, not the old temporary IP.

6. First-time server setup (Amazon Linux)

SSH into the server as ec2-user, then run:

```
# System updates (Amazon Linux uses dnf)
sudo dnf update -y

# Node.js 20 (matches GitHub Actions)
curl -fsSL https://rpm.nodesource.com/setup_20.x | sudo bash -
sudo dnf install -y nodejs git nginx

# PM2 globally
sudo npm install -g pm2

# Create app directory
mkdir -p ~/shopify_SAAS
mkdir -p ~/shopify_SAAS/server/logs
```

****No local Redis:**** Redis runs on ****Redis Cloud****. Do not install Redis on the EC2 instance. Connection details go in `server/.env` (see section 8).

6.1 Enable PM2 on reboot

```
pm2 startup
# Copy and run the command PM2 prints (starts with sudo env PATH=...)
```

7. Clone the project and install dependencies

```
cd ~/shopify_SAAS

# Clone (use HTTPS or deploy key - ask team for repo URL)
git clone https://github.com/YOUR_ORG/shopify_SAAS.git .

# Or if repo already exists on server:
# git pull origin main

cd server && npm install && cd ..
cd client && npm install && cd ..
```

8. Environment variables (create on the server)

.env files are not in Git and not deployed by GitHub Actions. You must create them directly on the EC2 server after SSH login.

GitHub Actions only passes EC2_HOST, EC2_USERNAME, and EC2_SSH_KEY for SSH deploy - it does not push app secrets (MONGO_URI, JWT_SECRET, Redis Cloud credentials, API keys, etc.). If .env is missing on the server, the app will fail even when deploy succeeds.

8.1 Server `.env` (required - create on server only)

SSH into the server and create ~/shopify_SAAS/server/.env:

```
nano ~/shopify_SAAS/server/.env
```

Example contents:

```
NODE_ENV=production
PORT=8000

# Database
MONGO_URI=mongodb+srv://USER:PASSWORD@cluster.mongodb.net/parallels?retryWrites=true&w=majority

# Auth
JWT_SECRET=your-long-random-secret

# Redis Cloud (from Redis Cloud dashboard - not localhost)
REDIS_HOST=your-redis-cloud-host.redis-cloud.com
REDIS_PORT=6379
REDIS_PASSWORD=your-redis-cloud-password

# Google OAuth & Ads
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_AD_DEVELOPER_TOKEN=
GOOGLE_REDIRECT_URI=https://parallels.messold.com/callback

# Shopify, Meta, Zoho, email, OpenAI, Apify, etc.
# Ask team lead for full list - see PROJECT_DOCUMENTATION.md $7

SERVER_ID=prod-1
```

Lock down permissions:

```
chmod 600 ~/shopify_SAAS/server/.env
```

Get `REDIS\_HOST`, `REDIS\_PORT`, and `REDIS\_PASSWORD` from your **Redis Cloud** database connection details. Whitelist the EC2 Elastic IP in Redis Cloud if required.

8.2 Client `.env.production` (required - create on server only)

Also create this on the server (used when building the frontend):

```
nano ~/shopify_SAAS/client/.env.production

VITE_API_URL=https://parallels.messold.com
VITE_LOCAL_API_URL=http://localhost:8000
VITE_APP_HANDLE=your-shopify-app-handle

chmod 600 ~/shopify_SAAS/client/.env.production
```

In production the frontend calls `VITE\_API\_URL` + `/api/...`. Nginx serves the SPA and proxies `/api` to the backend.

8.3 After updating `.env`

Restart PM2 so processes pick up changes:

```
cd ~/shopify_SAAS
pm2 restart ecosystem.config.js
```

If you changed client/.env.production, rebuild the frontend:

```
cd ~/shopify_SAAS/client && npm run build
sudo systemctl reload nginx
```

9. PM2 - ecosystem.config.js (two processes)

ecosystem.config.js is not in GitHub. Create it manually on the server at ~/shopify_SAAS/ecosystem.config.js (same as .env - GitHub Actions will not create or update this file).

It runs two PM2 processes:

PM2 name	Script	Purpose
`parallels-api`	`server/index.js`	Express API, Socket.IO, cron jobs
`metrics-worker`	`server/workers/metricsWorker.js`	BullMQ background metrics jobs

9.1 Create the file on the server

SSH into the server and create the file:

```
nano ~/shopify_SAAS/ecosystem.config.js
```

Paste this content exactly:

```
module.exports = {
  apps: [
    {
      name: 'parallels-api',
      cwd: './server',
      script: 'index.js',
      instances: 1,
      exec_mode: 'fork',
    }
  ]
}
```

```

env: {
  NODE_ENV: 'production',
  PORT: 8000,
},
max_memory_restart: '1G',
error_file: './logs/api-error.log',
out_file: './logs/api-out.log',
merge_logs: true,
time: true,
},
{
  name: 'metrics-worker',
  cwd: './server',
  script: 'workers/metricsWorker.js',
  instances: 1,
  exec_mode: 'fork',
  env: {
    NODE_ENV: 'production',
  },
  max_memory_restart: '1G',
  error_file: './logs/worker-error.log',
  out_file: './logs/worker-out.log',
  merge_logs: true,
  time: true,
},
],
};

```

Ensure the logs directory exists:

```
mkdir -p ~/shopify_SAAS/server/logs
```

9.2 Start processes (first time)

```

cd ~/shopify_SAAS
pm2 start ecosystem.config.js
pm2 save
pm2 status

```

9.3 View logs

```

pm2 logs parallels-api
pm2 logs metrics-worker
pm2 logs           # all processes

```

9.4 Restart (same as GitHub Actions deploy)

```

cd ~/shopify_SAAS
pm2 restart ecosystem.config.js

```

GitHub Actions runs `pm2 restart ecosystem.config.js` on deploy. If this file is missing on the server, deploy will fail at that step.

10. Build the frontend

```

cd ~/shopify_SAAS/client
npm run build

```

This creates client/dist/ - Nginx will serve these static files.

The GitHub Action copies `dist/\*` on deploy and reloads Nginx. For the **first deploy**, build locally on the server as above.

11. Nginx - reverse proxy, /api, and SSL

11.1 Nginx site config (HTTP only - before SSL)

Create /etc/nginx/conf.d/parallels.conf:

```
server {
    listen 80;
    server_name parallels.messold.com;

    root /home/ec2-user/shopify_SAAS/client/dist;
    index index.html;

    # React SPA - client-side routing
    location / {
        try_files $uri $uri/ /index.html;
    }

    # All API routes under /api
    location /api {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 300s;
        client_max_body_size 50M;
    }

    # Socket.IO (real-time notifications)
    location /socket.io {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

sudo nginx -t
sudo systemctl enable nginx
sudo systemctl start nginx
sudo systemctl reload nginx
```

11.2 Install SSL with Certbot (Let's Encrypt)

```
sudo dnf install -y certbot python3-certbot-nginx
sudo certbot --nginx -d parallels.messold.com
```

Follow prompts:

- Enter email for renewal notices
- Agree to terms
- Choose redirect HTTP -> HTTPS (recommended)

Certbot auto-updates the Nginx config and sets up renewal cron.

```
# Test renewal
sudo certbot renew --dry-run
```

11.3 After SSL - verify Nginx config

Certbot typically adds listen 443 ssl and certificate paths. Confirm:

```
sudo nginx -t
sudo systemctl reload nginx
```

11.4 Test endpoints

```
# API health (root of Express - not under /api)
curl -I https://parallels.messold.com/

# Example API route
curl -I https://parallels.messold.com/api/auth/check-token
```

12. GitHub Actions - automated deploy on push to main

Workflow file: `.github/workflows/deploy.yml`

On every push to main, GitHub:

1. SSHs into EC2
2. ``git pull origin main``
3. ``npm install`` in ``server/``
4. ``pm2 restart ecosystem.config.js``
5. ``npm install`` + copy ``client/dist`` + ``sudo systemctl reload nginx``

12.1 Required GitHub repository secrets

Go to GitHub repo -> Settings -> Secrets and variables -> Actions -> New repository secret:

Secret	Value
<code>`EC2_HOST`</code>	Elastic IP or domain, e.g. <code>`1.2.3.4`</code>
<code>`EC2_USERNAME`</code>	<code>`ec2-user`</code> (Amazon Linux)
<code>`EC2_SSH_KEY`</code>	<code>**Full contents**</code> of the <code>`pem`</code> file (including <code>`-----BEGIN/END-----`</code> lines)

These secrets are **only** for SSH deploy. Do **not** put ``MONGO_URI``, ``JWT_SECRET``, Redis Cloud credentials, or other app secrets in GitHub Actions - keep them in ``server/.env`` on the EC2 server.

12.2 Server must have git pull access

Ensure the EC2 instance can git pull without a password:

```
# Option A: HTTPS with credential helper (one-time login)
cd ~/shopify_SAAS
git config credential.helper store

# Option B: Deploy key (recommended for servers)
ssh-keygen -t ed25519 -C "ec2-deploy" -f ~/.ssh/github_deploy -N ""
cat ~/.ssh/github_deploy.pub

# Add this public key in GitHub -> Repo -> Settings -> Deploy keys (read-only)
```

12.3 Manual deploy test

Push a small change to main and watch Actions tab in GitHub. Or SSH and run the deploy script manually:

```
cd ~/shopify_SAAS
git pull origin main
cd server && npm install && cd ..
pm2 restart ecosystem.config.js
cd client && npm install && npm run build
sudo systemctl reload nginx
```

13. Verification checklist

Use this before handing off to the team:

- [] Amazon Linux 2023 instance running as `ec2-user`
 - [] Elastic IP attached and DNS A record points to it
 - [] SSH works with PEM key
 - [] `server/.env`, `client/.env.production`, and `ecosystem.config.js` created **on the server** (not in GitHub)
 - [] Redis Cloud credentials in `server/.env` and EC2 IP whitelisted in Redis Cloud
 - [] `pm2 status` shows `parallels-api` and `metrics-worker` **online**
 - [] `curl https://parallels.messold.com` returns the React app
 - [] `curl https://parallels.messold.com/api/...` hits the API (not 502)
 - [] SSL certificate valid in browser (padlock icon)
 - [] MongoDB connected (check `pm2 logs parallels-api` for DB errors)
 - [] GitHub Actions deploy succeeds on push to `main`
 - [] Login / OAuth callbacks work with production domain
-

14. Common commands

```
# SSH
ssh -i ~/path/to/key.pem ec2-user@ELASTIC_IP

# PM2
pm2 status
pm2 restart ecosystem.config.js
pm2 logs
pm2 monit

# Nginx
sudo nginx -t
sudo systemctl reload nginx
sudo systemctl status nginx

# App logs
tail -f ~/shopify_SAAS/server/logs/api-out.log
tail -f ~/shopify_SAAS/server/logs/worker-out.log

# Pull latest manually
cd ~/shopify_SAAS && git pull origin main
```

15. Troubleshooting

502 Bad Gateway on /api

- Check API is running: ``pm2 status``
- Check port: ``curl http://127.0.0.1:8000/`` from the server
- Check logs: ``pm2 logs parallels-api``

WebSocket / notifications not working

- Confirm Nginx has ``/socket.io`` proxy block with ``Upgrade`` headers
- Check CORS in ``server/config/socket.js`` includes your production domain

Certbot fails

- DNS must resolve to this server before running certbot
- Port 80 must be open in Security Group

GitHub Actions deploy fails on SSH

- Verify ``EC2_HOST``, ``EC2_USERNAME``, ``EC2_SSH_KEY`` secrets
- PEM key must include newlines exactly as in the file
- Security Group must allow SSH from GitHub Actions IPs (or use a self-hosted runner)

metrics-worker crashes

- Check Redis Cloud credentials in ``server/.env`` (``REDIS_HOST``, ``REDIS_PORT``, ``REDIS_PASSWORD``)
- Confirm EC2 Elastic IP is allowed in Redis Cloud network/access settings
- Check ``pm2 logs metrics-worker``

Frontend shows old version after deploy

- Rebuild: ``cd client && npm run build``
- Reload nginx: ``sudo systemctl reload nginx``
- Hard refresh browser (Ctrl+Shift+R)

Quick reference

Item	Value
OS	Amazon Linux 2023 (<code>`ec2-user`</code>)
App path on server	<code>`~/home/ec2-user/shopify_SAAS`</code>
API port (internal)	<code>`8000`</code>
Public API path	<code>`https://parallels.messold.com/api/*`</code>
Redis	Redis Cloud (external)
PM2 config	<code>`~/shopify_SAAS/ecosystem.config.js`</code> (**create on server**)
Nginx config	<code>`/etc/nginx/conf.d/parallels.conf`</code>
Server env	<code>`~/shopify_SAAS/server/.env`</code> (**create on server**)
Client env	<code>`~/shopify_SAAS/client/.env.production`</code> (**create on server**)
Deploy trigger	Push to <code>`main`</code> branch

Document generated for Parallels (Shopify SAAS) - Messold